

An AI-Powered Lost and Found Matching Algorithm Using Image Embedding Generation and Semantic Similarity Analysis

Chapters 1–3

System: FindIt

Institution: AMA University

CHAPTER 1

THE PROBLEM AND ITS BACKGROUND

1.1 Introduction

Campus communities lose personal property every day. Phones, earphones, IDs, calculators, keys, bags, and similar items are left in classrooms, libraries, laboratories, and lounges. Most universities still reconnect owners and finders through desk logs, word-of-mouth, or social media posts. Those channels do not scale well: descriptions are inconsistent, search is manual, and follow-up depends on office hours and chance.

The core technical weakness is comparison. A finder may turn in a photo of a black backpack; an owner may type a short note with different wording. Staff must mentally bridge that gap. Keyword search cannot, because it rewards shared strings rather than shared meaning. A stronger approach is to map images and text into a common numerical space, then rank likely pairs by similarity.

This thesis develops that approach as **FindIt**, a web-based institutional lost-and-found matcher. FindIt encodes submissions with CLIP ViT-B/32, stores 512-dimensional embeddings in PostgreSQL with pgvector, and ranks candidates using cosine similarity plus a weighted multimodal scoring policy. Users receive an ordered shortlist above confidence thresholds and confirm a claim before the system treats the pair as accepted and sends coordination email. The research contribution is the matching algorithm and the surrounding ticket workflow, not a redesign of physical storage hardware.

1.2 Background of the Study

Vision–language models trained with contrastive objectives learn to place related images and captions near one another in embedding space. CLIP is a widely used instance of that idea: an image encoder and a text encoder produce aligned vectors so cosine similarity becomes a practical relevance score. The ViT-B/32 configuration offers a balance of quality and runtime suitable for a campus prototype.

Lost-and-found matching is a natural application. Evidence is often mixed—sometimes only a photo, sometimes only text, sometimes both—and the same object can appear under different lighting, angles, or phrasing. An algorithm that can score cross-modal and same-modal pairs, filter weak scores, and present ranked options gives users a usable retrieval experience while keeping final acceptance under human control.

1.3 Statement of the Problem

Institutional lost-and-found units lack an automated method to compare lost reports with found inventory when evidence is visual, textual, or both. Manual and keyword-based processes are slow and error-prone,

especially as the number of tickets grows.

This study therefore asks:

1. How can image and text submissions be represented so that lost and found tickets become comparable in a single similarity framework?
2. How should multimodal similarity signals be combined, filtered, and ranked for campus tickets?
3. How can ranked AI output be connected to a practical confirmation and notification workflow without silently linking the wrong items?

1.4 Objectives of the Study

1.4.1 General Objective

To develop an AI-powered lost-and-found matching algorithm that uses image embedding generation and semantic similarity analysis to improve automated retrieval of corresponding lost and found items in an institutional setting.

1.4.2 Specific Objectives

1. Design a ticket-based web reporting system for lost and found submissions with image and text fields, category, location, date, and contact information.
2. Generate normalized image and text embeddings for each ticket using CLIP ViT-B/32 and persist them for retrieval.
3. Implement a semantic similarity matcher that fuses available multimodal scores, applies contextual adjustments, assigns confidence tiers, and returns a ranked candidate list.
4. Support bidirectional matching (lost against found inventory; found against open lost reports) and a confirmation-centered claim flow with email coordination.

1.5 Significance of the Study

End users. Students and staff gain a single channel to file reports and review likely matches instead of relying only on repeated desk visits or scattered posts.

Lost-and-found offices. Staff receive structured tickets and machine-ranked candidates, reducing time spent on exhaustive manual comparison.

Academic contribution. The study documents an applied multimodal ranking pipeline—embedding generation, score fusion, thresholds, shortlist policy, and human confirmation—for institutional item recovery.

1.6 Scope and Delimitation

In scope. FindIt web client; FastAPI services; PostgreSQL/pgvector storage; CLIP ViT-B/32 encoding; multimodal ranking with tiers and trimming; claim confirmation; email/outbox notifications for claim events; desktop and mobile browser access.

Delimitation. The system does not perform live camera surveillance, GPS tracking of items, or automated physical locker control. Matching quality depends on submission quality and on the pretrained encoder. Legal ownership disputes remain outside the software. Prototype mail may use a local outbox unless SMTP is configured.

1.7 Conceptual Framework

FindIt follows an Input–Process–Output flow.

Stage	Content
Input	Lost/found tickets (text, images, category, location, date, email); optional account credentials; claim confirmation actions
Process	Media handling; CLIP encoding; cosine similarity across available signals; weighted fusion; category/location/date adjustments; tiering and ranked trimming; claim state updates; mail dispatch
Output	Stored embeddings and tickets; ranked match shortlists; claim records; coordination emails; queue visibility for operations

Human confirmation is part of the process: AI retrieves and ranks; users accept or reject before inventory is closed.

1.8 Definition of Terms

FindIt — The prototype platform implementing the proposed matching algorithm and ticket workflows.

CLIP ViT-B/32 — Contrastive vision–language model used to embed images and text into a shared 512-d space.

Embedding — A numeric vector representing an image or a text string.

Cosine similarity — Score of alignment between normalized embeddings.

Multimodal fusion — Combining multiple CLIP-derived similarities into one ranking score.

Confidence tier — Strong / possible / weak label derived from the final score.

Ranked shortlist — Ordered candidates shown after thresholding, allowing selection beyond rank 1.

Claim confirmation — User acceptance that links a lost ticket to a found ticket and starts coordination notices.

CHAPTER 2

REVIEW OF RELATED LITERATURE AND STUDIES

2.1 Institutional Lost-and-Found Operations

Published accounts of university lost-and-found practice repeatedly describe brittle documentation and weak verification when processes are manual or informal (Mullins & Lee, 2017; Tan & Chong, 2023; Castro et al., 2022). The operational lesson for this thesis is specific: recovery quality is limited by the institution's ability to compare heterogeneous evidence quickly and consistently.

2.2 Digital Lost-and-Found Applications

Web and mobile lost-and-found systems improve intake, categorization, and alerts (Kim et al., 2019; Gupta & Sharma, 2020; Pandey et al., 2020; Castro et al., 2022; Shrivastava et al., 2025; Salman, 2022). Their common ceiling is matching logic. Many remain keyword- or staff-driven after the form is submitted. FindIt treats digital ticketing as necessary infrastructure and focuses research effort on embedding-based ranking between tickets.

2.3 Image Embedding and Vision–Language Similarity

Contrastive language–image training produces encoders that support cross-modal retrieval with cosine similarity (Radford et al., 2021; Peng, 2025). Vision Transformer backbones such as ViT-B/32 are standard image towers in this family. Extensions examine efficiency, robustness, and finer representations (Li et al., 2022; Fang et al., 2022; Cui et al., 2022; Dong et al., 2026). For FindIt, these results justify CLIP as the feature generator. They also caution against treating raw similarity as a final authority when campus photos are noisy and many items look alike.

2.4 Ranking, Thresholds, and Human-in-the-Loop Decisions

Information retrieval practice favors ranked lists and early-rank metrics when queries are ambiguous (Muennighoff et al., 2022). Short user text worsens lexical mismatch (Amur et al., 2023). A lost-and-found product should therefore (a) discard weak scores, (b) show an ordered shortlist, and (c) require confirmation before notifying parties and closing records. That policy converts imperfect ranking into usable recovery support.

2.5 Synthesis

Prior work supplies three ingredients FindIt combines: institutional need for better comparison, digital tickets/notifications as the service shell, and CLIP-style embeddings as the similarity engine. The gap is an end-to-end campus matcher that defines multimodal fusion, thresholds, bidirectional search, ranked

user choice, and confirmation-triggered coordination. Chapter 3 specifies the technical platform used to close that gap.

CHAPTER 3

TECHNICAL REQUIREMENTS AND SYSTEM ARCHITECTURE

3.1 Overview

FindIt is a three-service prototype: a static web frontend, a FastAPI application that runs CLIP and business logic, and PostgreSQL with pgvector for relational data and embeddings. Docker Compose orchestrates local deployment.

3.2 Hardware Requirements

3.2.1 User Devices

Requirement	Minimum expectation
Device	Laptop, desktop, or smartphone
Browser	Current Chromium, Firefox, Safari, or Edge
Media	Ability to upload item photos
Network	Stable internet path to the API

3.2.2 Development / Demo Host

Requirement	Minimum expectation
CPU	Multi-core
RAM	8 GB minimum; 16 GB recommended
GPU	Optional CUDA for faster CLIP encoding
Disk	Space for database, uploads, model cache, mail outbox

3.3 Software Requirements

3.3.1 Frontend

- Static FindIt UI (HTML/JavaScript) served by nginx
- Flows: register/login, report lost, report found, review ranked matches, confirm claims, inspect queues

3.3.2 Backend and Data

Component	Choice
Language	Python 3
API	FastAPI + Uvicorn
Database	PostgreSQL 16
Vectors	pgvector (vector(512))
Migrations	Alembic
Auth	JWT; email verification before registered login
Media	Validated/resized uploads on the API host

3.3.3 Matching Engine

Component	Choice
Model	OpenAI CLIP ViT-B/32 (PyTorch)
Similarity	Cosine (dot product on L2-normalized vectors)
Fusion	Adaptive weights over available image-image and text-image signals

Context	Category multiplier; location and date boosts
Decision aids	Tiers at approximately 0.85 / 0.70 / 0.55; ranked trimming that keeps top candidates for user choice
Categories	Prompt-based CLIP suggestion over a fixed campus category list

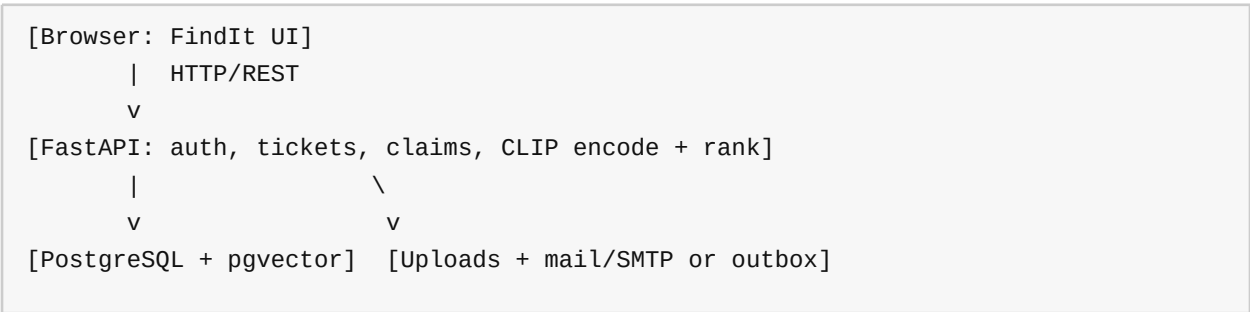
3.3.4 Notifications and Deployment

- Claim lifecycle email (accepted / cancelled / processed)
- SMTP when configured; otherwise filesystem mail outbox
- Docker Compose services: api, postgres, frontend

3.4 Peopleware

Role	Responsibility
Developer	Implements UI, API, matching, and claim workflow
Evaluator	Tests ranking behavior and end-to-end claims
Demo operator	Runs containers and mail/outbox checks
Desk staff (pilot)	Performs physical handoff after digital confirmation

3.5 System Architecture



Lost path. Encode ticket → score available found items → return tiered ranked shortlist → user may confirm a candidate (including non-top ranks) → claim emails on confirmation workflow.

Found path. Encode ticket → reverse-score open lost items → return ranked shortlist in-app → claim/confirm as applicable.

Safeguards. Near-duplicate found uploads can be rejected; self-matches by the same email are excluded; weak scores never appear in the shortlist.

3.6 Functional Capability Summary

1. Ticket intake for lost and found items
2. Embedding generation and persistent vector storage
3. Bidirectional semantic matching with multimodal fusion
4. Thresholded ranked presentation for human selection
5. Claim confirmation and email coordination for desk pickup

These capabilities define the technical baseline against which later methodology and evaluation chapters measure the proposed algorithm.